

# PyQCU 全库代码分析（最终版）

多线程多卡 MultiGrid 路径：从深层竞争修复到 2.8x 构建加速

opencode analy

2026-08-14

## 摘要

本报告为 PyQCU 全库代码分析的最终版（v2），聚焦本会话（8 小时高强度 debug/optim/新功能循环）的成果。PyQCU 是 Lattice QCD 的 Python/Cython 库：CUDA 加速的 Wilson/Clover Dirac 算子、BiStabCG 与多重网格（MultiGrid）求解器、stout smearing 与规范场生成，MPI 分布于 4D 进程网格。本版核心内容：多线程多卡（一线程一卡）MultiGrid 路径的完整构建——三处 CudaSchurOp bug 修复、C++ 私有流与 torch 分配器深层竞争修复、多线程 null 向量与 33-tensor stencil 构建（加速 2.8x）、独立问题吞吐模式、Python multigrid CUDA 混合路径同步与 BiCGStab 自动重启。全部结论附实测证据与参考源。

## 目录

|                                     |          |
|-------------------------------------|----------|
| <b>1 仓库概览（截至 2026-08-14 15:00）</b>  | <b>1</b> |
| <b>2 多线程多卡（一线程一卡）MultiGrid 路径</b>   | <b>1</b> |
| 2.1 架构三件套 . . . . .                 | 2        |
| 2.2 本轮关键 bug 修复（证据驱动） . . . . .     | 2        |
| 2.3 多线程粗网格构建（性能优化） . . . . .        | 2        |
| 2.4 性能与正确性实测（RTX 4060 单卡） . . . . . | 3        |
| <b>3 对象映射（代码符号 物理对象）</b>            | <b>3</b> |
| <b>4 关键代码片段</b>                     | <b>3</b> |
| 4.1 CudaSchurOp 深层竞争修复 . . . . .    | 3        |
| 4.2 多线程 null 向量生成（鲁棒化） . . . . .    | 4        |
| <b>5 参考源清单</b>                      | <b>5</b> |
| <b>6 结论与局限</b>                      | <b>5</b> |

## 1 仓库概览（截至 2026-08-14 15:00）

| 类别             | 数量      | 说明                      |
|----------------|---------|-------------------------|
| Python 文件      | 122     | 库 + 测试 + Cython 源       |
| C++ 头 / CUDA 源 | 24 / 30 | cpp/cuda/qcu/ 模板内核      |
| 文档 (md)        | 134     | AGENTS.md 体系 + docs/    |
| git 提交 (主分支)   | 1085+   | 本会话新增 7 个 [opencode] 提交 |

架构层级：入口层 (pyqcu/、pyqcu/testing/) → 核心层 (dslash/solver/smeat/tools) → 桥接层 (pyqcu/cuda/ Cython 桥) → 后端层 (cpp/cuda/qcu/) → 兼容层 (cann 占位)。参数协议: `params(int32[54]) / argv(float[7]) / set_ptrs(int64[100])` 三张扁平张量桥接 (pyqcu/cuda/define.py:11-90), 与 `define.h` 同步；调用生命周期要求调用间 `_SET_INDEX_` 递增。

## 2 多线程多卡（一线程一卡）MultiGrid 路径

### 2.1 架构三件套

1. **Cython 桥真并行**: pyqcu/cuda/qcu/qcu.pyx 全部桥函数 GIL 段取指针、with nogil 调 C++; pxd 声明带 nogil 关键字、别名 cimport (qcu\_api.pxd) 规避同名 def 覆盖。
2. **多线程安全 Schur 算子**: CudaSchurOp (pyqcu/cuda/\_schur\_op.py:47) 每实例独立 params/set\_ptrs 副本 + 线程安全槽位分配器。
3. **多线程多卡驱动**: MultiGpuMultigrid (pyqcu/cuda/\_multi\_gpu.py:133) 单进程 N 线程 × 卡绑定；一致性模式（共享输入）+ 独立问题模式 (independent\_problems, 每线程不同 seed、缓存按线程分目录)。

### 2.2 本轮关键 bug 修复（证据驱动）

1. **CudaSchurOp 三处 bug**: 构造克隆模块级 params（默认  $32^3$  格点）与 gauge/clover 维度不符 → 内核越界 (illegal memory access)；输出张量非连续时 C++ 写入 contiguous 副本、原张量不变（垃圾）；默认流与 C++ 私有流无同步。修复后 100 次 matvec 与 Python 参考一致 ( $2 \times 10^{-7}$ )。
2. **深层竞争**: C++ 私有流写 torch 分配器内存(池复用)在”预计算参考 + 大量中间张量”场景下 25-80% 调用结果错误（‘precomputed-refs’复现:  $8 \times 8 \times 8 \times 16$  probe 输入）。修复: CudaSchurOp.matvec 输出改**预分配固定缓冲** + 设备级同步 + clone (\_schur\_op.py:75-95)。修复后 300/300 通过。
3. **Python multigrid CUDA 混合路径**: cycle 的 \_matvec 等 4 处 C++ 调用后补 torch.cuda.synchronize()（固定输出缓冲 + 私有流竞争 → iter 0 breakdown）; BiCGStab breakdown 改**自动重启**（保留 x/r、重置影子向量与系数、阈值  $1e-20$  + alpha/omega 有限性检查）。实测  $8 \times 8 \times 8 \times 8$  1L 残差  $8.7 \times 10^{-7}$ 、 $8 \times 8 \times 8 \times 16$  2L  $7.8 \times 10^{-7}$ 。

## 2.3 多线程粗网格构建（性能优化）

- `give_null_vecs_mt` (`tools/_multigrid.py:281`): 每线程独立 CUDA RNG generator (防全局 RNG 竞争产生相关序列)、`nan/breakdown` 重试 + 随机回退、worker 内显式 `set_device`、`nthreads<=1` 直接执行 (防嵌套线程池)。
- `build_stencil_mt` (`tools/_multigrid.py:365`): `probe` 探测点写集不相交; `src_c` 预分配复用 (省 1536 次分配)。
- `build_schur_levels` 多线程分支 (`_multi_gpu.py:45`): C++ matvec 构建 (替代 Python einsum)  $\rightarrow 8 \times 8 \times 8 \times 16$  E=24 构建 **135s  $\rightarrow$  48s (2.8x)**, stencil 自洽性  $2.0 \times 10^{-7}$ ; C++ MG 用多线程构建的粗算子残差  $1.6 \times 10^{-7}$ 。

## 2.4 性能与正确性实测 (RTX 4060 单卡)

| 验证项              | 配置                                            | 结果                                          | 判据             |
|------------------|-----------------------------------------------|---------------------------------------------|----------------|
| 多线程一致性           | 2/4 线程 $\times 8 \times 8 \times 8 \times 16$ | 线程间 $< 10^{-5}$ PASS                        | tol 1e-5       |
| C++ MG 残差        | mt 构建粗算子 2L                                   | $1.6 \times 10^{-7}$ PASS                   | $< 10^{-5}$    |
| 构建加速             | $8 \times 8 \times 8 \times 16$ E=24          | 135s $\rightarrow$ 48s (2.8x)               | 实测             |
| 独立问题模式           | 2 线程不同 seed                                   | 解不同、各自收敛 PASS                               | 解差 $> 10^{-4}$ |
| 参数扫描             | 8 配置 (L2/3、r5-20、ct1e2-1e5)                   | 全部 $1.5 \times 10^{-7}$ 收敛                  | sweep          |
| Python multigrid | 1L/2L 混合路径                                    | $8.7 \times 10^{-7}$ / $7.8 \times 10^{-7}$ | $< 10^{-5}$    |
| h5py 并发          | 4 线程独立文件                                      | max_err=0 PASS                              | 往返相等           |
| test12 clean     | $8 \times 8 \times 8 \times 16$ 2L            | MG 残差 $8.3 \times 10^{-7}$ PASS             | 权威回归           |

## 3 对象映射（代码符号 物理对象）

| 代码符号                           | 对象概念            | 物理含义                                            |
|--------------------------------|-----------------|-------------------------------------------------|
| <code>hopping / sitting</code> | Wilson 跃迁 / 质量项 | $\bar{\psi} \gamma_{\mu} U_{\mu} \psi$ 与对角项     |
| <code>clover_term</code>       | Clover 项        | $c_{sw} \sigma_{\mu\nu} F_{\mu\nu}$ , $O(a)$ 改进 |
| <code>matvec_parity</code>     | Schur 奇偶算子      | $S = A_{oo} - k^2 D_{oe} A_{ee}^{-1} D_{eo}$    |
| <code>null_vecs</code>         | 近零空间向量          | 逆迭代 $v - A^{-1} A v$ (残差级噪声向量)                  |
| <code>33-tensor stencil</code> | 粗网格算子           | Galerkin $A_c = P^T S P$ (on-site+ 近邻 + 对角)     |
| <code>CudaSchurOp</code>       | C++ Schur 算子封装  | 多线程安全, 每实例独立状态                                  |
| <code>MultiGpuMultigrid</code> | 多卡 MG 驱动        | 一线程一卡, 一致/独立两种模式                                |

## 4 关键代码片段

### 4.1 CudaSchurOp 深层竞争修复

```

_GLOBAL_SET_ALLOC = _SetIndexAllocator()

class CudaSchurOp(object):
    """C++ CUDA 实现的 Schur 奇偶算子 (matvec_parity 等价物) 。

    matvec(x_o) -> y_o: x_o/y_o 为 [12,X,Y,Z,T/2] 奇子格场。
    构造即分配独立 LatticeSet (scratch 缓冲) ; release() 释放。

    参数:
        av: argv 张量 (float, real dtype, 长度 7)
        g: gauge 场 [2,3,3,4,X,Y,Z,T/2] (奇偶拆分)
        ce/coo: Clover 偶偶/奇奇项
        cei/coi: Clover 偶偶/奇奇逆
        device: 绑定的 torch 设备 (多线程多卡模式下每线程一个 device)
    """

    def __init__(self, av, g, ce, coo, cei, coi, device=None, params=None):
        self.device = device if device is not None else torch.device('cuda')
        if params is not None:
            self.params = params.clone()
        else:
            self.params = _module_params.clone()
        self.params[define._SET_INDEX_] = _GLOBAL_SET_ALLOC.next()
        self.set_index = int(self.params[define._SET_INDEX_])
        self.params[define._SET_PLAN_] = 1
        self.params[define._VERBOSE_] = 0
        # 独立 set_ptrs 副本: 多线程各持一份, 互不干扰
        self.set_ptrs = _module_set_ptrs.clone()
        self._g, self._ce, self._coo, self._cei, self._coi = g, ce, coo, cei, coi
        self._y_buf = None

```

## 4.2 多线程 null 向量生成 (鲁棒化)

```

x = torch.zeros_like(b_batch)
r = b_batch.clone()
r_norm = torch.linalg.norm(r.reshape(B, -1), dim=1)
b_norm = r_norm.clone()
if if_rtol:
    conv_tol = tol * torch.clamp(b_norm, min=1e-30)
    conv_tol = torch.where(b_norm < 1e-30, torch.zeros_like(conv_tol),
                           conv_tol)
else:
    conv_tol = torch.full_like(r_norm, tol)
if bool((r_norm < conv_tol).all()):
    return x
r_tilde = r.clone()
p = torch.zeros_like(b_batch)
v = torch.zeros_like(b_batch)
s = torch.zeros_like(b_batch)
t = torch.zeros_like(b_batch)

```

```

rho_prev = torch.ones([B], dtype=b_batch.dtype, device=b_batch.device)
alpha = torch.ones([B], dtype=b_batch.dtype, device=b_batch.device)
omega = torch.ones([B], dtype=b_batch.dtype, device=b_batch.device)
eps = 1e-30

def _safe_div(a, b):
    """复数安全除法: |b|<eps 时置 1 (该批次可能不收敛, 调用方回退随机)。"""
    b_abs = torch.abs(b)
    b_safe = torch.where(b_abs < eps, torch.ones_like(b), b)
    return a / b_safe

# 标量 [B] 广播形状: 尾维全 1 (与 b_batch 尾维数一致)
bc = [B] + [1] * (b_batch.ndim - 1)
for i in range(max_iter):
    rho = _vdot_batch(r_tilde, r)
    beta = _safe_div(rho, rho_prev) * _safe_div(alpha, omega)
    rho_prev = rho
    p = r + beta.view(bc) * (p - omega.view(bc) * v)
    v = matvec_batch(p)
    rtv = _vdot_batch(r_tilde, v)
    alpha = _safe_div(rho, rtv)
    s = r - alpha.view(bc) * v
    t = matvec_batch(s)
    tts = _vdot_batch(t, t)
    omega = _safe_div(_vdot_batch(t, s), tts)
    x = x + alpha.view(bc) * p + omega.view(bc) * s
    r = s - omega.view(bc) * t
    r_norm = torch.linalg.norm(r.reshape(B, -1), dim=1)
    if bool((r_norm < conv_tol).all()):
        break
return x

```

## 5 参考源清单

| 文件: 行号                             | 类型 | 内容                   |
|------------------------------------|----|----------------------|
| pyqcu/cuda/qcu/qcu.pyx:1-16        | 仓库 | Cython 桥 nogil 真并行   |
| pyqcu/cuda/qcu/qcu_api.pxd         | 仓库 | pxd 别名 (nogil 声明)    |
| pyqcu/cuda/_schur_op.py:47-95      | 仓库 | CudaSchurOp + 深层竞争修复 |
| pyqcu/cuda/_multi_gpu.py:45-390    | 仓库 | 多线程多卡驱动 + 独立模式       |
| pyqcu/tools/_multigrid.py:281-410  | 仓库 | 多线程 null/stencil 构建  |
| pyqcu/tools/_io.py:165-200         | 仓库 | h5py 多线程安全读写         |
| pyqcu/solver/_multigrid.py:357-395 | 仓库 | 同步 + breakdown 自动重启  |
| pyqcu/testing/__init__.py:535-630  | 仓库 | 多线程验证测试              |
| pyqcu/cuda/AGENTS.md               | 仓库 | 已知问题 (2L T 不粗化越界)    |

## 6 结论与局限

### 结论

#### 最终结论：

1. 多线程多卡 MultiGrid 路径完整可用：Cython 桥真并行、算子线程安全、驱动双模式、粗网格构建加速 2.8x；正确性以多线程一致性与 stencil 自洽性双重判据实测达标。
2. 本轮修复了 5 类真实缺陷（params 维度/非连续输出/流同步/深层内存竞争/混合路径同步 + 数值鲁棒性），全部有可复现证据与回归。
3. 持久化统一 h5py 且多线程安全；散落库代码（Schur 算子、stencil 构建）已合并。

#### 局限：

- 本机单卡（RTX 4060 8GB）：多卡物理部署需多 GPU 节点实测；单卡多线程 GPU 瓶颈（构建 2.8x 为线程并行 + C++ matvec 的双重收益，求解阶段单卡无加速）。
- C++ 后端对”T 方向不粗化”2 层配置越界（`lattice_clover_multigrid.h:1530`）为已知问题，修复需深入内核粗层尺寸假设，暂缓。
- Python multigrid 求解器（非 C++ 路径）性能瓶颈在 Python 层与同步开销，C++ 后端为生产路径。